

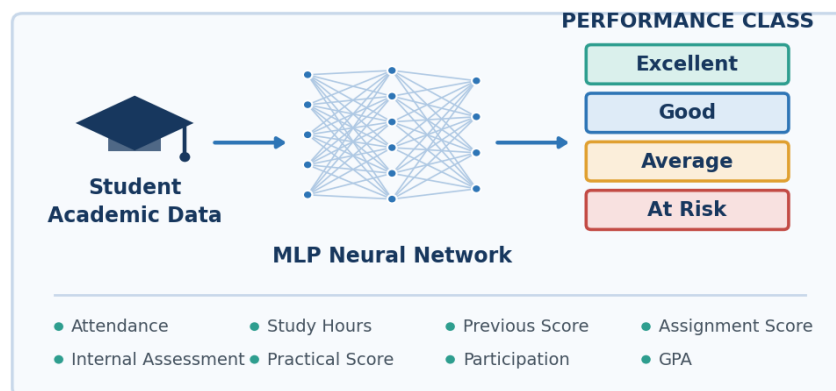


GOVERNMENT TOOL ROOM AND TRAINING CENTRE, BELAGAVI

DEPARTMENT OF DIPLOMA IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PROJECT REPORT

AI-BASED STUDENT PERFORMANCE PREDICTION AND EARLY ACADEMIC RISK DETECTION SYSTEM



PARTICULARS	DETAILS
Project Title	AI-Based Student Performance Prediction and Early Academic Risk Detection System
Submitted By	Abhinandan Jadhav
USN / Roll Number	8080110
Department	Diploma in Artificial Intelligence and Machine Learning
Semester	5th Semester
Guide / Faculty	K. S. Shirur
Institution	Government Tool Room and Training Centre, Belagavi
Academic Year	2026

TABLE OF CONTENTS

Certificate	3
Declaration	4
Acknowledgement	5
Abstract	6
1. Introduction	7
2. Problem Statement and Objectives	9
3. Existing System and Proposed System.....	11
4. System Requirements and Technologies	13
5. Dataset and Input Features	15
6. System Architecture	17
7. Methodology and Algorithm	20
8. Implementation and User Interface.....	23
9. Results and Model Evaluation	26
10. Conclusion, Future Scope and References	29

CERTIFICATE

This is to certify that the project work entitled **“AI-Based Student Performance Prediction and Early Academic Risk Detection System”** is a bonafide record of the work carried out by **Abhinandan Jadhav**, bearing USN / Roll Number **8080110**, of **5th Semester**, Department of **Diploma in Artificial Intelligence and Machine Learning, Government Tool Room and Training Centre, Belagavi**, in partial fulfilment of the requirements for the award of the degree of **Diploma in Artificial Intelligence and Machine Learning** during the academic year **2026**.

The project has been carried out under my supervision and guidance. The work embodied in this report is original and has not been submitted, in part or in full, to any other institution or university for the award of any degree or diploma.

The candidate has satisfactorily completed all the requirements prescribed for the project work, and the report has been approved as it satisfies the academic requirements in respect of the project work.

Signature of the Guide

Signature of the Head of Department

DECLARATION

I, **Abhinandan Jadhav**, bearing Roll Number **8080110**, a student of **5th Semester**, Department of **Diploma in Artificial Intelligence and Machine Learning**, **Government Tool Room and Training Centre, Belagavi**, hereby declare that the project work entitled “**AI-Based Student Performance Prediction and Early Academic Risk Detection System**” has been independently carried out by me under the guidance of **K. S. Shirur**, Department of **Diploma in Artificial Intelligence and Machine Learning**, during the academic year **2026**.

I further declare that the content presented in this report is the result of my own effort and study. The design, implementation, and analysis described herein have been developed by me, and all external sources of information, tools, libraries, and published material referred to during the course of this work have been duly acknowledged in the References section.

To the best of my knowledge and belief, this report contains no material previously published or written by another person, nor material which has been accepted for the award of any other degree or diploma of any university or institution, except where due acknowledgement has been made in the text.

I also declare that no experimental results, evaluation metrics, or dataset statistics have been reproduced without being generated by the implemented system.

Abhinandan Jadhav
Roll Number: 8080110

Date: _____

ACKNOWLEDGEMENT

The successful completion of any project depends largely on the encouragement, guidance, and support extended by many individuals. I take this opportunity to express my sincere gratitude to everyone who contributed to the completion of this project work.

I express my heartfelt thanks to [Principal / Director Name], Government Tool Room and Training Centre, Belagavi, for providing the academic environment, infrastructure, and computing facilities necessary to carry out this project.

I am deeply grateful to [HOD Name], Head of the Department of Diploma in Artificial Intelligence and Machine Learning, for the constant encouragement and for extending the departmental resources required at every stage of the work.

I wish to record my profound gratitude to my project guide, K. S. Shirur, [Designation], Department of Diploma in Artificial Intelligence and Machine Learning, whose valuable suggestions, technical insight, and continuous supervision shaped the direction of this work. The guidance received on model design, data preprocessing, and evaluation methodology was instrumental in bringing this project to its present form.

I also thank the teaching and non-teaching staff of the Department of Diploma in Artificial Intelligence and Machine Learning for their cooperation and timely assistance throughout the development of the system.

Finally, I extend my sincere appreciation to my family and friends for their patience, motivation, and moral support during the course of this project.

ABSTRACT

Academic performance monitoring in most institutions remains largely retrospective: a student's difficulty is recognised only after examination results are declared, by which

time corrective intervention has limited value. This project presents an AI-Based Student Performance Prediction and Early Academic Risk Detection System, a software-only solution that uses a deep learning model to classify a student's expected academic performance from routinely available academic indicators, so that support can be initiated before the outcome is fixed.

The system accepts eight input features — Attendance, Study Hours, Previous Score, Assignment Score, Internal Assessment, Practical Score, Participation and GPA. These features are cleaned and normalised using StandardScaler from Scikit-learn, and are then supplied to a Multi-Layer Perceptron (MLP) neural network implemented with TensorFlow and Keras. The network employs ReLU activation in its hidden layers and a Softmax output layer, and is trained using Cross-Entropy Loss with the Adam optimizer. The model classifies each student into one of four performance classes: At Risk, Average, Good and Excellent.

The complete solution is delivered through an interactive Streamlit dashboard built in Python, supported by NumPy and Pandas for data handling. The result is a lightweight, deployable decision-support tool that converts routine academic records into early, actionable insight for students, faculty and academic administrators.

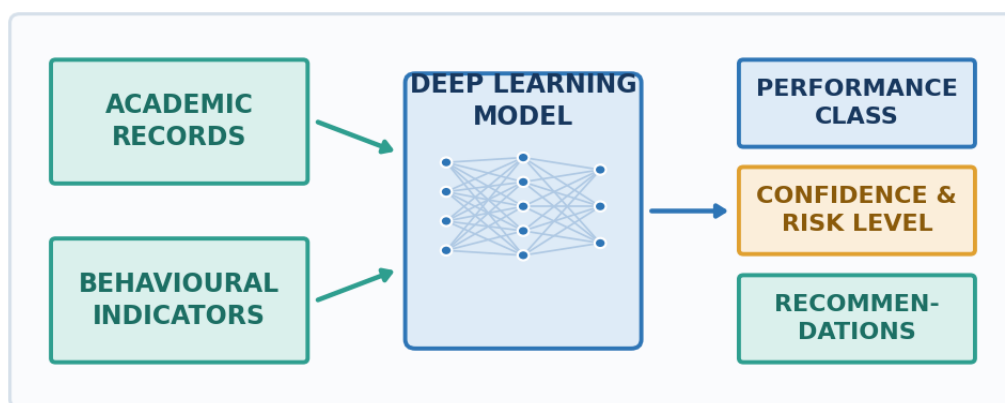


Figure 2: End-to-End Concept of the AI-Based Student Analytics System

1. INTRODUCTION

1.1 Background

Educational institutions generate a continuous stream of academic data — attendance registers, internal assessment marks, assignment submissions, practical evaluations, participation records and cumulative grade point averages. Although this data is collected systematically, it is seldom analysed in a predictive manner. It is typically used for reporting after the fact, rather than for anticipating which students are likely to struggle.

Artificial Intelligence, and in particular deep learning, is well suited to this gap. A neural network can learn the combined, non-linear influence of several academic indicators on a student's eventual performance — relationships that are difficult to capture through manual inspection or simple threshold rules such as a fixed attendance cut-off.

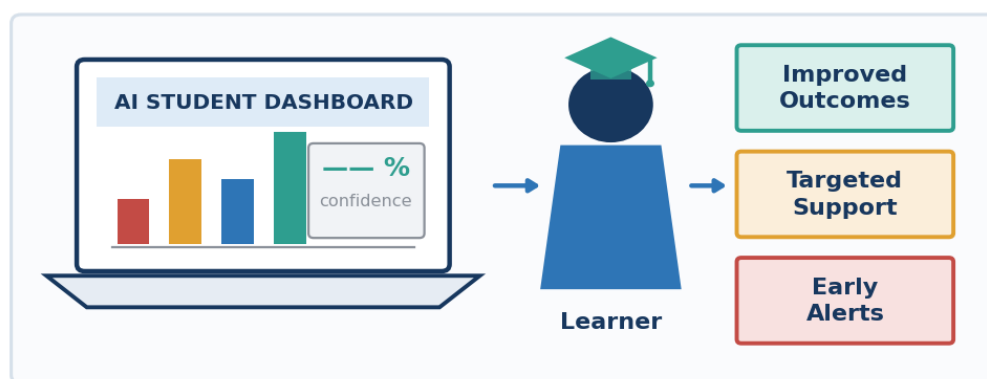


Figure 3: Student Interaction with an AI-Assisted Academic Support System

1.2 Motivation

The motivation for this project arises from a practical observation: a student who is heading towards failure usually displays measurable warning signs well before the final examination. Declining attendance, weak internal assessment marks, poor assignment performance and low participation are visible indicators. What is missing is a mechanism that combines these signals, weighs them appropriately, and issues an early, confidence-qualified alert.

1.3 Overview of the Proposed Work

This project develops a software-only system that performs exactly this function. A Multi-Layer Perceptron is trained on student academic records and deployed behind an interactive Streamlit dashboard. For a given set of inputs the system returns the predicted performance class, the confidence associated with that prediction, an interpreted risk level, a probability distribution across all four classes, and a set of recommendations. The system requires no additional hardware, sensors or external devices; it operates entirely on data that institutions already maintain.

1.4 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 defines the problem statement and objectives, and Chapter 3 contrasts the existing practice with the proposed system. Chapter 4 lists the requirements and technologies, and Chapter 5 describes the dataset and input features. Chapters 6 and 7 present the system architecture and the methodology, Chapter 8 the implementation and user interface, Chapter 9 the results, and Chapter 10 the conclusion and future scope.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Existing academic monitoring practices are reactive rather than predictive. Student performance is assessed after examinations are conducted, which means that students at academic risk are identified only at a stage where remedial action can no longer influence the outcome. Faculty members must manually review multiple records — attendance, internal marks, assignments and practicals — to form a judgement, a process that is time-consuming, subjective, difficult to scale across large cohorts, and inconsistent between evaluators.

Furthermore, conventional monitoring offers no measure of certainty. A student is either flagged or not flagged, with no indication of how strongly the available evidence supports that judgement, and no ranked view of which outcomes are plausible. There is therefore a need for an automated, data-driven system that predicts student performance in advance, expresses the prediction together with a confidence value and an associated risk level, and translates the result into concrete recommendations.

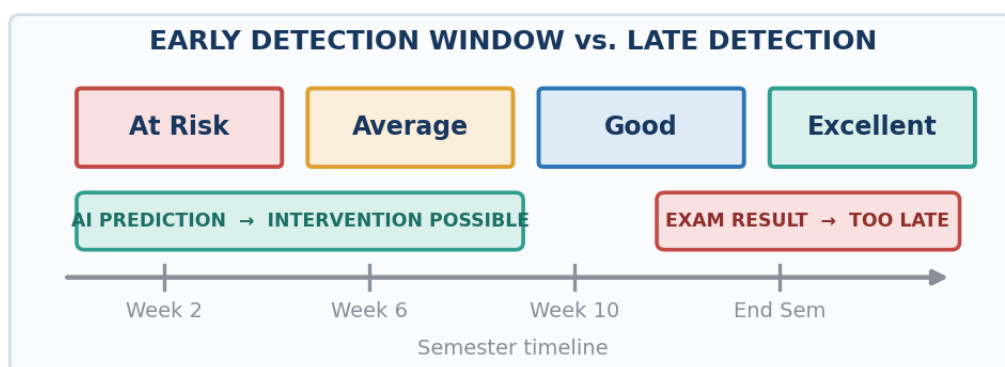


Figure 4: Early Academic Risk Detection versus Post-Examination Assessment

2.2 Objectives

1. To design and develop an AI-based system that predicts student academic performance using routinely available academic indicators.
2. To classify each student into one of four performance classes: At Risk, Average, Good or Excellent.
3. To detect academic risk at an early stage, before final examination results are declared.
4. To implement a Multi-Layer Perceptron (MLP) neural network using TensorFlow and Keras with ReLU activation and a Softmax output layer.
5. To apply systematic data preprocessing and feature normalisation using StandardScaler so that all eight input features contribute on a comparable scale.
6. To report a confidence percentage and an interpreted risk level alongside every prediction.
7. To visualise the class probability distribution so that the relative likelihood of each performance class is transparent.
8. To generate actionable recommendations aligned with the predicted class and the student's input profile.
9. To evaluate the trained model using standard classification metrics and training result visualisations.
10. To deploy the complete solution as an interactive Streamlit web dashboard requiring no specialised hardware.

2.3 Scope of the Project

The scope of this project is limited to the software implementation of a predictive academic decision-support tool. The system operates on structured numerical academic data supplied through the dashboard interface. It does not involve any hardware components, embedded devices, sensors or network equipment. The predictions produced by the system are intended to support, not replace, the academic judgement of faculty members.

3. EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 Existing System

In the existing practice, student performance is tracked manually. Faculty members maintain attendance registers and mark sheets, and periodically inspect them to identify students who appear to be underperforming. Assessment is largely post-examination, and the evaluation of risk depends on individual experience and intuition.

- Manual review of mark sheets and attendance registers across multiple documents.
- Assessment performed after examinations, leaving no window for corrective action.
- No quantified confidence associated with any judgement of risk.
- Uniform, generic feedback that is not tailored to an individual student's weak areas.
- Difficult to scale as cohort size increases; prone to oversight and inconsistency.
- Records maintained largely in spreadsheets, with no predictive capability.

3.2 Proposed System

The proposed system replaces manual inspection with an automated deep learning model. Eight academic features are normalised and passed to a trained Multi-Layer Perceptron, which outputs a probability distribution over four performance classes. The class with the highest probability is reported as the prediction, the corresponding probability is reported as the confidence percentage, and a risk level is derived from the result. Recommendations are generated to accompany the prediction, and the entire interaction takes place through a Streamlit dashboard.

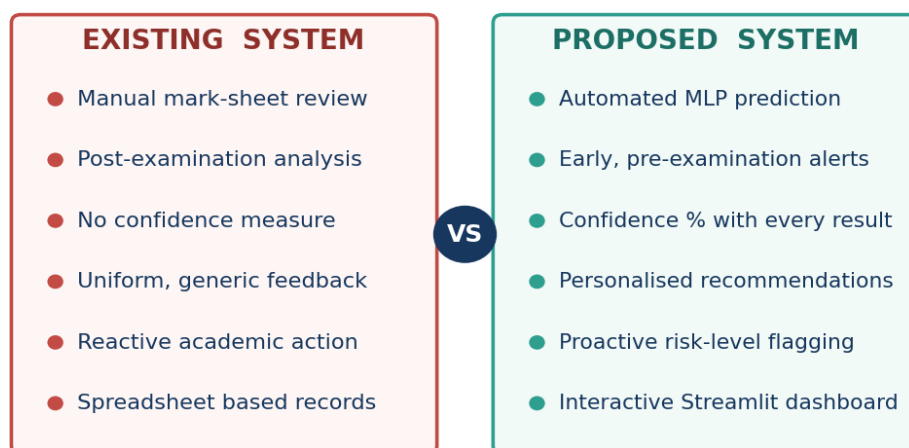


Figure 5: Traditional Academic Monitoring versus AI-Powered Predictive Analytics

3.3 Comparative Analysis

PARAMETER	EXISTING SYSTEM	PROPOSED SYSTEM
Method	Manual inspection of records	Automated MLP neural network prediction
Timing	After examination results	Early, before final examinations
Confidence	Not available	Confidence percentage with every prediction
Risk Level	Subjective judgement	Derived systematically from model output
Output Detail	Pass / fail style assessment	Four-class prediction with full probability distribution
Feedback	Generic and uniform	Recommendations aligned to the student profile
Scalability	Limited by manual effort	Scales to large cohorts without added effort
Interface	Spreadsheets and registers	Interactive Streamlit dashboard
Evaluation	No model validation	Standard metrics and training visualisations

Table 3: Comparison between the Existing and Proposed Systems

4. SYSTEM REQUIREMENTS AND TECHNOLOGIES

This project is implemented entirely in software. No microcontrollers, sensors, actuators, embedded boards or other hardware peripherals are used at any stage of development or deployment. The requirements below describe a standard computing environment adequate for training and running the model.

4.1 Hardware Requirements (Standard Computing Environment)

COMPONENT	SPECIFICATION
Processor	Intel Core i3 or higher (or equivalent AMD processor)
RAM	4 GB minimum; 8 GB recommended for model training
Storage	Sufficient free disk space for the Python environment, dataset and saved model
Display	Standard monitor capable of rendering the web dashboard
Graphics	Not mandatory; the model can be trained and executed on CPU

Table 4: Hardware Requirements

4.2 Software Requirements

COMPONENT	SPECIFICATION
Operating System	Windows, Linux or macOS
Programming Language	Python
Deep Learning Framework	TensorFlow with the Keras API
Machine Learning Library	Scikit-learn (including StandardScaler)
Numerical Computation	NumPy
Data Handling	Pandas
Web Application Framework	Streamlit
Development Environment	Any Python-compatible IDE or code editor
Browser	Any modern web browser for accessing the Streamlit dashboard

Table 5: Software Requirements

SOFTWARE TECHNOLOGY STACK

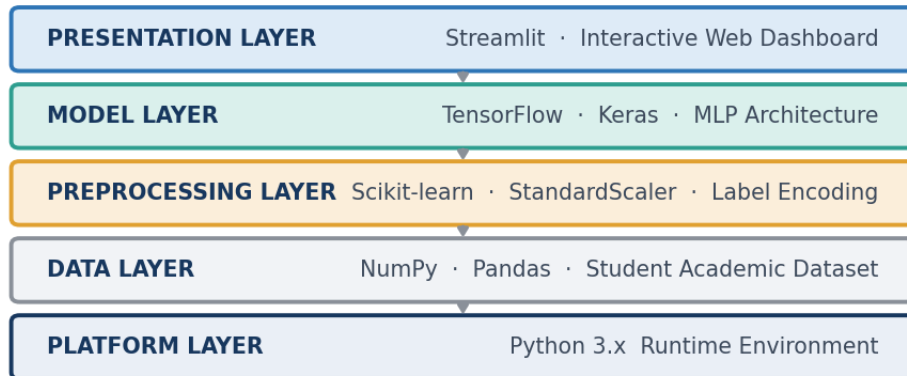


Figure 6: Software Technology Stack of the Proposed System

4.3 Technologies Used

TECHNOLOGY	ROLE IN THE PROJECT
Python	Core programming language used for the complete implementation
Streamlit	Builds the interactive web dashboard for input, prediction and visualisation
TensorFlow / Keras	Defines, compiles, trains and serves the MLP neural network
NumPy	Numerical array operations and efficient mathematical computation
Pandas	Loading, structuring and manipulating the student academic dataset
Scikit-learn	Preprocessing utilities, dataset splitting and evaluation metrics
StandardScaler	Normalises the eight input features to zero mean and unit variance
MLP (Multi-Layer Perceptron)	Feed-forward neural network architecture used for classification
ReLU Activation	Introduces non-linearity in the hidden layers of the network
Softmax Output	Converts output-layer scores into a probability distribution over four classes
Cross-Entropy Loss	Loss function optimised during multi-class classification training
Adam Optimizer	Adaptive gradient-based optimisation algorithm used to update network weights

Table 6: Technologies Used and Their Roles

5. DATASET AND INPUT FEATURES

5.1 Dataset Description

The system is trained on a structured dataset of student academic records. Each record consists of eight numerical academic indicators together with a labelled performance class. The dataset is loaded and organised using Pandas, converted into numerical arrays using NumPy, and split into training and testing subsets before model training begins.

Dataset size, number of records, class distribution and train–test split ratio should be inserted here from the actual project dataset. Example: “[Insert actual value from the project dataset]”.

5.2 Input Features

The following eight features form the input vector supplied to the neural network. Each feature represents a distinct and independently observable dimension of academic behaviour or achievement.

SL.	INPUT FEATURE	DESCRIPTION
1	Attendance	Extent of the student’s presence in scheduled classes
2	Study Hours	Time devoted by the student to independent study
3	Previous Score	Performance recorded in earlier examinations or terms
4	Assignment Score	Marks obtained for submitted assignments
5	Internal Assessment	Marks obtained in internal or continuous assessment tests
6	Practical Score	Performance in laboratory and practical evaluations
7	Participation	Level of active involvement in classroom and academic activities
8	GPA	Cumulative grade point average reflecting overall standing

Table 7: Student Performance Input Features

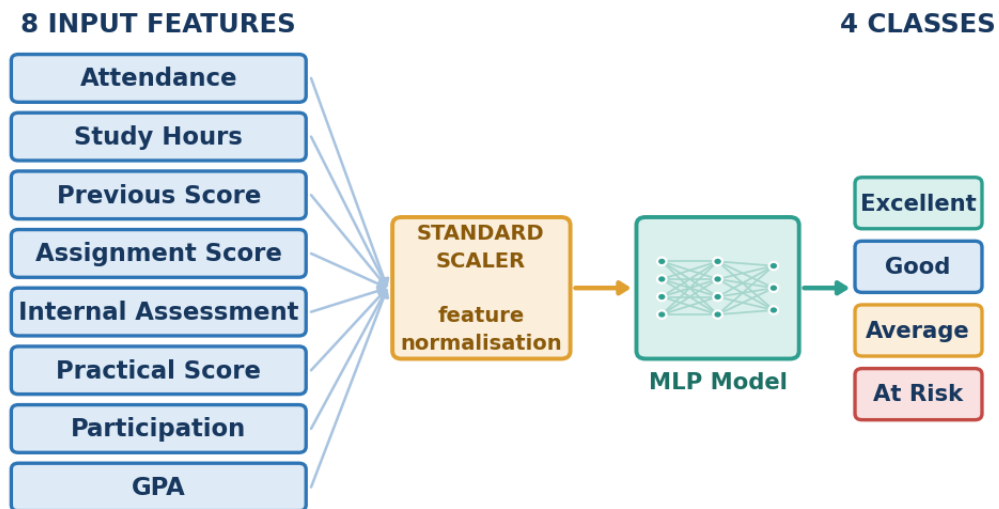


Figure 7: Input Features, Feature Scaling and Output Performance Classes

5.3 Output Classes

PERFORMANCE CLASS	INTERPRETATION
At Risk	Indicators point to a high likelihood of academic difficulty; immediate intervention is advised
Average	Performance is adequate but improvable; targeted support is beneficial
Good	Consistent performance across most indicators; steady progress expected
Excellent	Strong performance across indicators; enrichment and advanced work are appropriate

Table 8: Performance Classification Categories

5.4 Data Preprocessing

- Loading the dataset into a structured Pandas DataFrame.
- Handling missing or inconsistent entries prior to training.
- Encoding the performance class labels into a form suitable for multi-class classification.
- Applying StandardScaler so that every feature has zero mean and unit variance, preventing features with larger numerical ranges from dominating the learning process.
- Splitting the data into training and testing subsets so that model performance is measured on unseen records.

6. SYSTEM ARCHITECTURE

The system follows a linear, layered processing pipeline. Raw student academic data enters at the top of the pipeline and is progressively transformed into a prediction, a confidence value, a risk level and a set of recommendations, all of which are finally presented through the Streamlit dashboard. Each stage has a single, well-defined responsibility, which keeps the design modular and the data flow easy to trace.

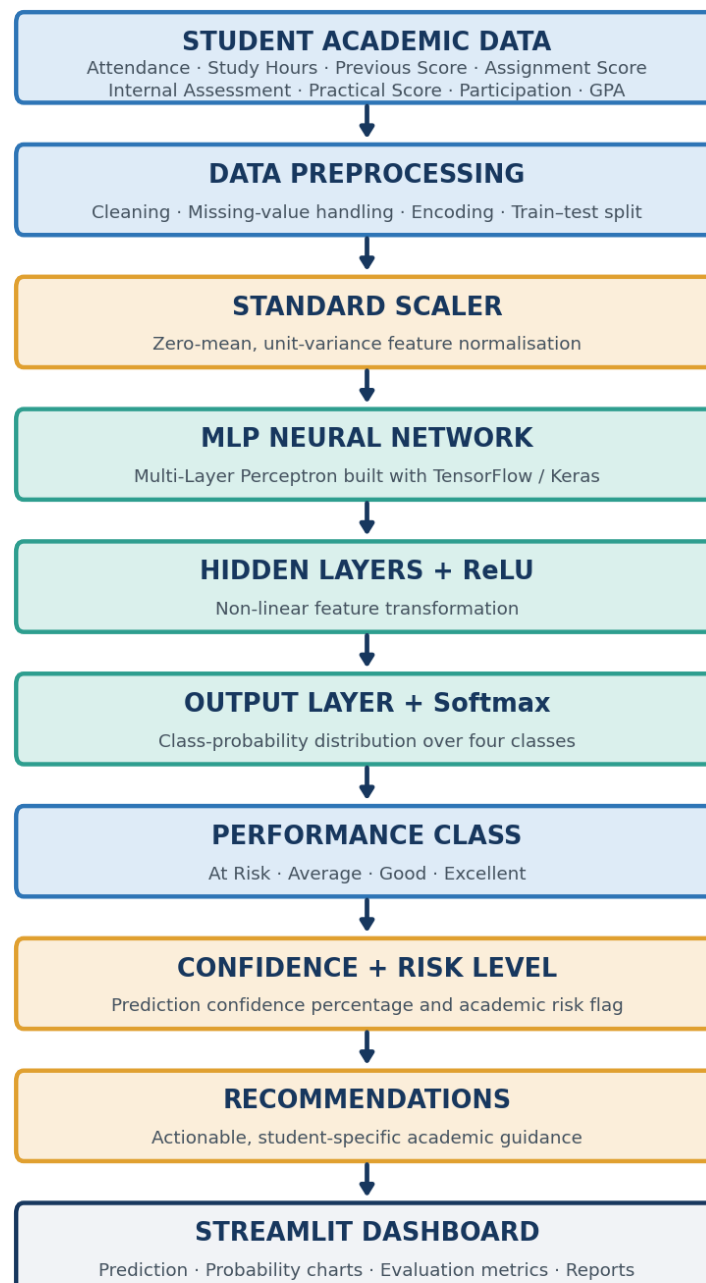


Figure 8: System Architecture of the AI-Based Student Performance Prediction System

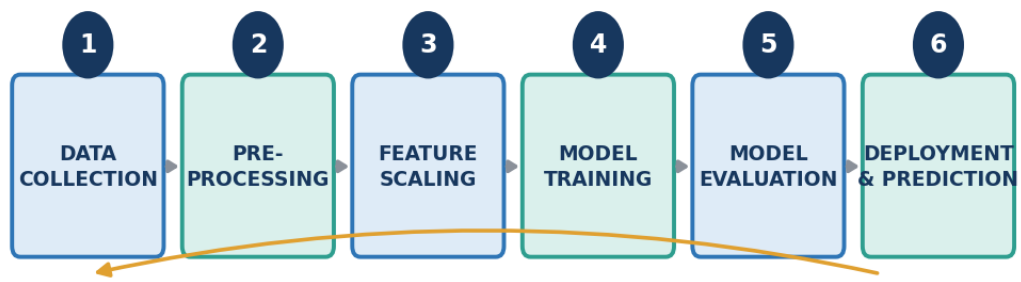
6.1 Description of Architectural Stages

STAGE	FUNCTION
Student Academic Data	Collects the eight input features from the dataset or from dashboard input
Data Preprocessing	Cleans the data, handles missing values, encodes labels and splits the dataset
Standard Scaler	Normalises all features to zero mean and unit variance
MLP Neural Network	Feed-forward network built with TensorFlow and Keras that performs classification
Hidden Layers + ReLU	Apply non-linear transformations that let the network learn complex feature interactions
Output Layer + Softmax	Produces a normalised probability distribution across the four performance classes
Performance Class	Reports the class with the highest probability as the predicted outcome
Confidence + Risk Level	Reports the associated probability as confidence and derives the academic risk level
Recommendations	Generates guidance aligned with the predicted class and the input profile
Streamlit Dashboard	Presents predictions, probability charts, metrics and visualisations to the user

Table 9: Functional Description of Architectural Stages

6.2 Data Flow

Data flows in a single forward direction through the pipeline. There is no feedback loop during inference: once the model is trained, a prediction request passes through preprocessing, scaling and the network in one pass. Feedback occurs only during the training phase, where the loss computed by the Cross-Entropy function is propagated backwards and the Adam optimizer updates the network weights across successive epochs.



Iterative refinement — retraining and hyper-parameter tuning

Figure 9: Prediction Workflow from Data Collection to Deployment

7. METHODOLOGY AND ALGORITHM

7.1 Methodology

The development methodology proceeds in six sequential phases. Data is first collected and preprocessed, then normalised, after which the neural network is trained, evaluated and finally deployed through the dashboard. Where evaluation indicates scope for improvement, the model is retrained with adjusted hyper-parameters before redeployment.

7.2 MLP Model Structure

The classifier is a Multi-Layer Perceptron: a fully connected, feed-forward neural network. The input layer accepts the eight standard-scaled academic features. One or more hidden layers apply ReLU activation, introducing the non-linearity required to model interactions between features — for example, the combined effect of low attendance together with a weak internal assessment score. The output layer contains four neurons and applies the Softmax function, converting the raw output scores into a probability distribution in which all four values are non-negative and sum to one.

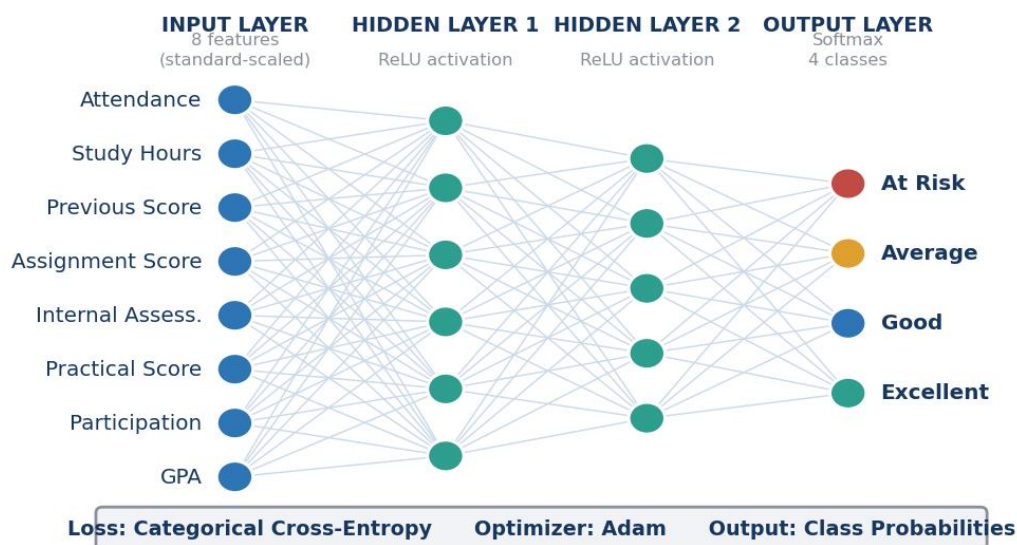


Figure 10: Structure of the Multi-Layer Perceptron (MLP) Model

Note: the number of neurons shown in the hidden layers of Figure 10 is representative and is intended to illustrate the architecture. The actual layer configuration should be taken from the implemented model definition.

7.3 Activation Functions

ReLU is applied in the hidden layers because it is computationally inexpensive and does not saturate for positive inputs, which allows gradients to propagate effectively during training. Softmax is applied at the output layer because it converts the four raw output scores into a normalised probability distribution, so that the predicted class and its associated confidence can be read directly from the same output. Sigmoid and Tanh are shown alongside them for comparison; both saturate at their extremes, which makes them less suitable for the hidden layers of this network.

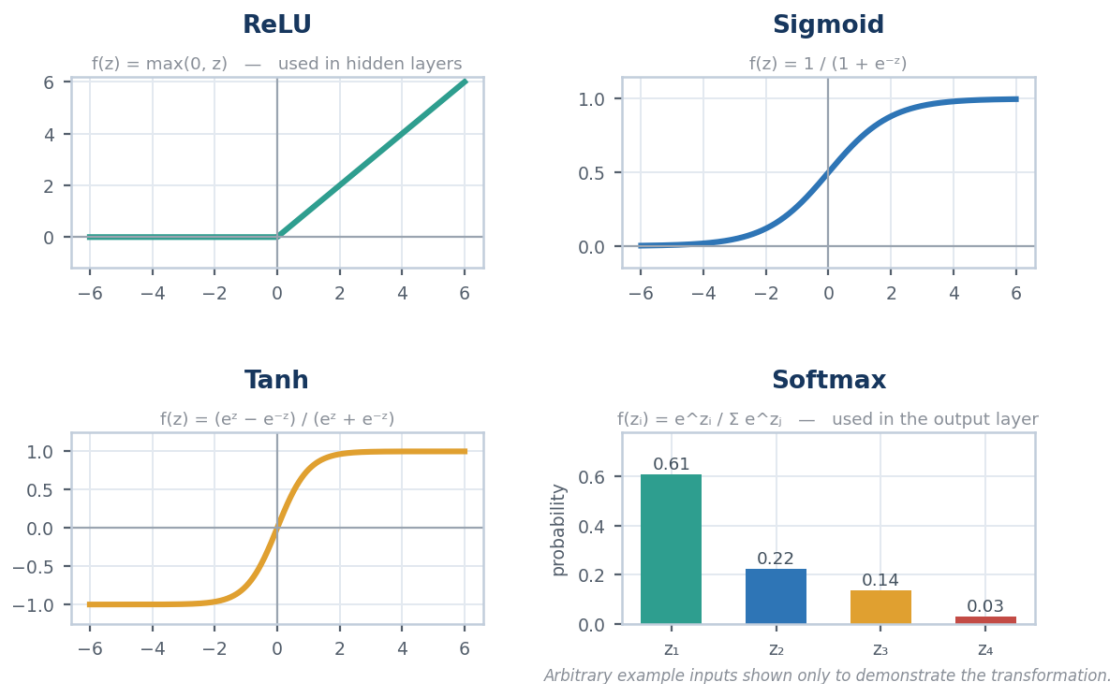


Figure 11: Activation Functions — ReLU and Softmax as used in the model, with Sigmoid and Tanh for comparison

7.4 Training Configuration

PARAMETER	SPECIFICATION
Model Type	Multi-Layer Perceptron (feed-forward neural network)
Framework	TensorFlow with the Keras API
Input Layer	8 standard-scaled academic features
Hidden Layer Activation	ReLU (Rectified Linear Unit)
Output Layer Activation	Softmax over 4 performance classes
Loss Function	Cross-Entropy Loss
Optimizer	Adam
Feature Scaling	StandardScaler (Scikit-learn)
Number of Hidden Layers	2
Neurons per Hidden Layer	64 ,32
Learning Rate	0.001
Batch Size	32
Number of Epochs	100

Table 10: Model and Training Configuration

7.5 Algorithm

1. Load the student academic dataset using Pandas.
2. Clean the data and handle missing or inconsistent values.
3. Separate the eight input features from the performance class label.
4. Encode the class labels into a numerical representation suitable for multi-class classification.
5. Split the dataset into training and testing subsets.
6. Fit StandardScaler on the training data and transform both the training and testing feature sets.
7. Define the MLP architecture in Keras with ReLU-activated hidden layers and a Softmax output layer of four neurons.
8. Compile the model with Cross-Entropy Loss and the Adam optimizer.
9. Train the model over the configured number of epochs, recording loss and accuracy for each epoch.
10. Evaluate the trained model on the test subset and compute the classification metrics.

8. IMPLEMENTATION AND USER INTERFACE

8.1 Implementation Overview

The system is implemented in Python and organised into functional modules: a data handling and preprocessing module built on Pandas, NumPy and Scikit-learn; a model module that defines, trains and saves the Keras MLP; a prediction module that applies the fitted StandardScaler and performs inference; a recommendation module that maps the predicted class and input profile to guidance; and a presentation module built with Streamlit.

The trained model and the fitted scaler are persisted after training, so that the dashboard performs inference without retraining on each run. This keeps response time low and ensures that every prediction is produced by the same validated model.

8.2 User Interface

The Streamlit dashboard presents all eight input features as interactive controls. On submission, the interface displays the predicted performance class prominently, together with the confidence percentage and the interpreted risk level. A chart shows the probability assigned to each of the four classes, allowing the user to see how close the decision was. A recommendations panel lists the suggested actions, and a metrics section reports the evaluation results for the trained model.

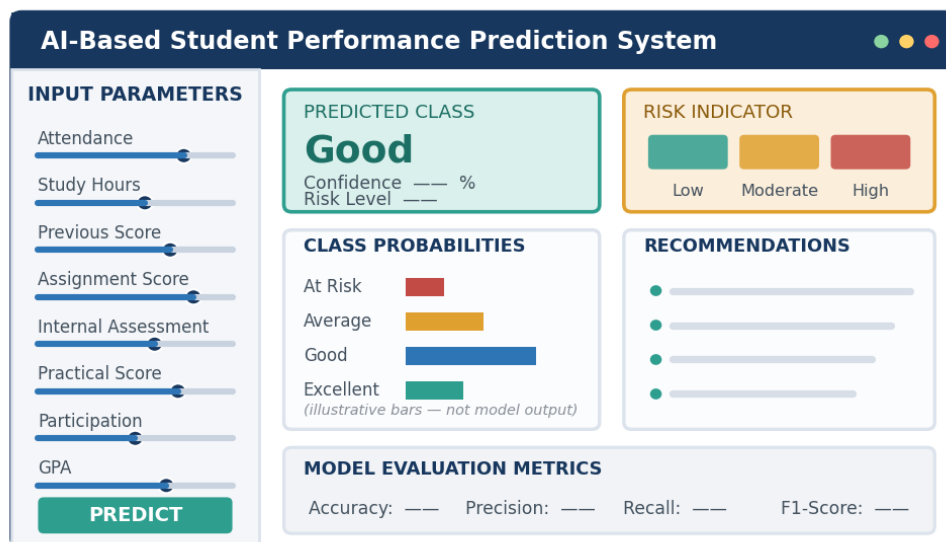


Figure 12: Representative Layout of the Streamlit Prediction Dashboard (illustration, not an application screenshot)

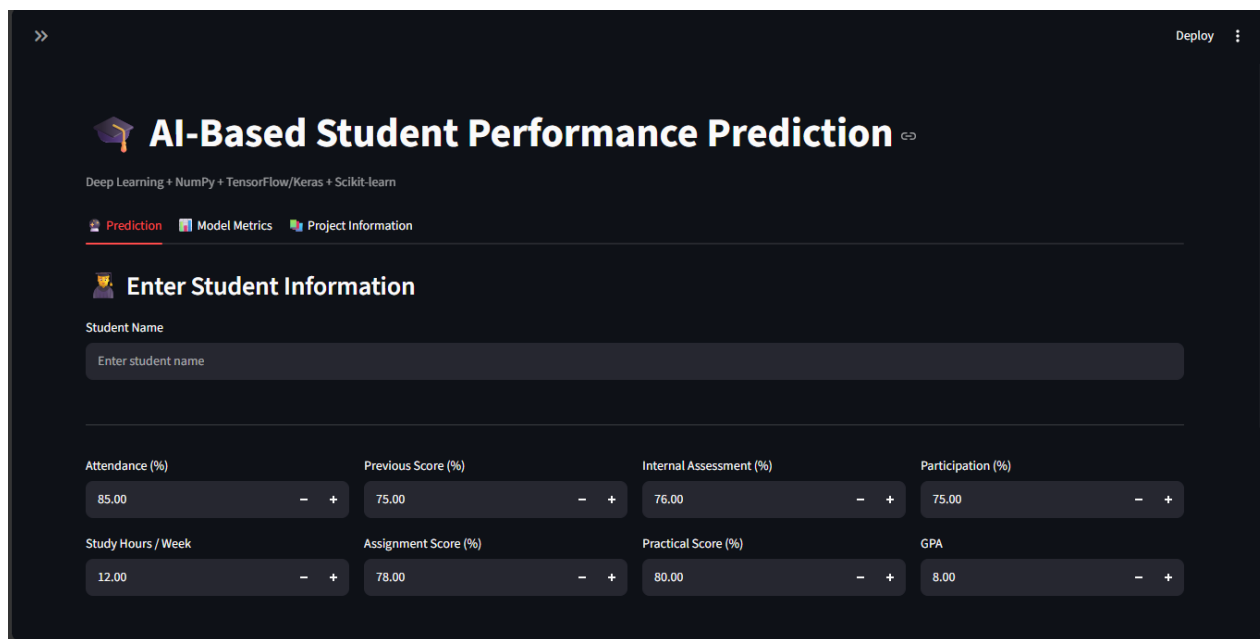


Figure 13: Reserved Frames for Screenshots of the Running Streamlit Application

8.3 Interface Modules

MODULE	FUNCTION
Input Panel	Accepts values for all eight academic input features
Prediction Display	Shows the predicted performance class for the supplied inputs
Confidence Indicator	Reports the probability associated with the predicted class as a percentage
Risk Level Indicator	Communicates the interpreted academic risk level
Probability Visualisation	Charts the Softmax probability assigned to each of the four classes
Recommendations Panel	Lists guidance derived from the prediction and the input profile
Evaluation Metrics View	Presents the classification metrics obtained for the trained model
Training Visualisations	Displays the training result graphs generated during model development

Table 12: Functional Modules of the Streamlit Interface

9. RESULTS AND MODEL EVALUATION

9.1 Evaluation Approach

The trained model is evaluated on the held-out test subset, which contains records not seen during training. Performance is assessed using standard multi-class classification metrics, supported by training-history curves and a confusion matrix that together indicate whether the model generalises rather than memorises.

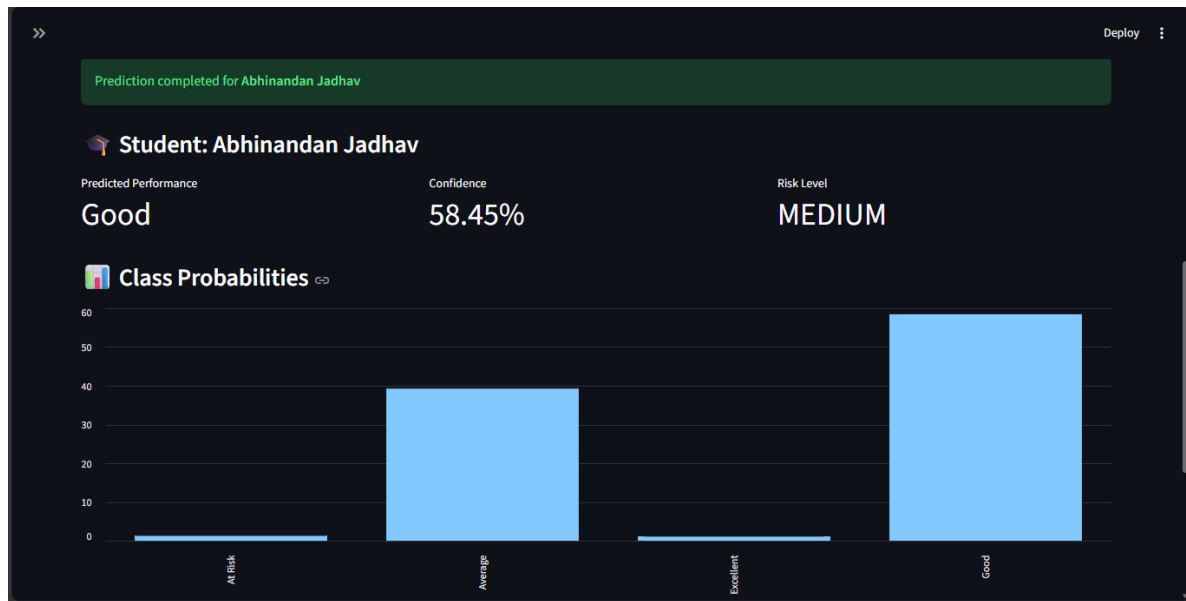


Table 13: Model Evaluation Metrics (values to be inserted from the trained model)

9.4 Training Result Visualisations

The following result images are generated during model training and evaluation. The activation function comparison is reproduced in Figure 11 of Chapter 7, since it depends only on the mathematical definitions of the functions and not on the trained model. The remaining five images depend entirely on the dataset and the trained model; they were not supplied with the project materials, and no substitute curves have been drawn, since fabricated graphs would misrepresent the behaviour of the model. Each frame in Figure 14 should be replaced with the corresponding image file from the project directory.

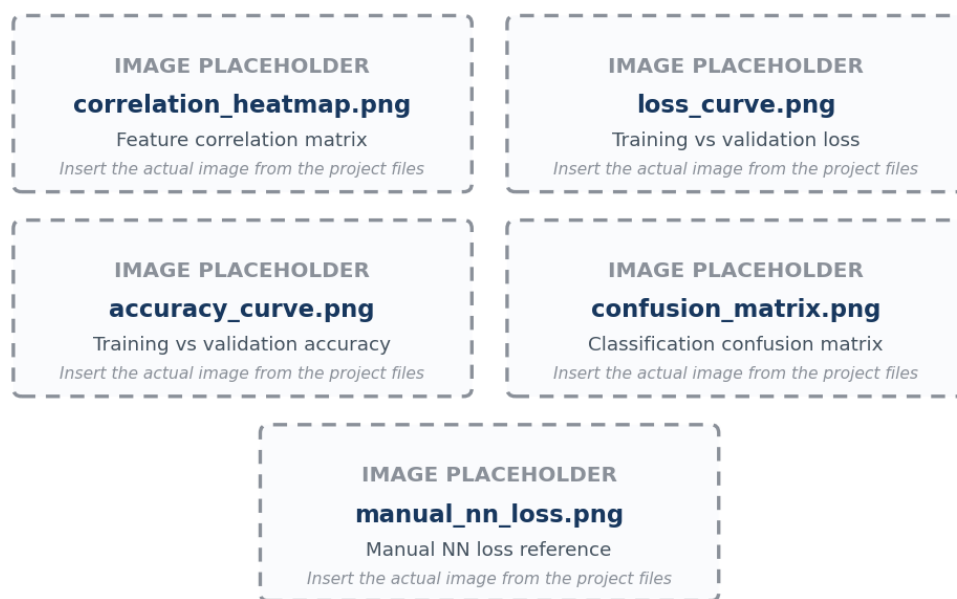


Figure 14: Placeholders for the Remaining Training and Evaluation Result Visualisations

10. FUTURE SCOPE , CONCLUSION AND REFERENCES

10.1 Future Scope

- Extending the system to accept batch inputs so that an entire class or cohort can be evaluated in a single operation.
- Adding an interpretability layer that identifies which input features contributed most strongly to a given prediction.
- Tracking predictions over successive assessment cycles to reveal whether a student's trajectory is improving or declining.
- Integrating the system with institutional academic databases so that features are populated automatically.
- Exploring alternative and deeper neural architectures, and comparing them against the current MLP baseline.
- Introducing role-based views so that students, faculty and administrators each see an appropriate level of detail.
- Generating exportable reports summarising predictions and recommendations for academic review meetings.
- Expanding the recommendation module with a wider, curriculum-aligned set of guidance rules.

10.2 Conclusion

This project has presented an AI-Based Student Performance Prediction and Early Academic Risk Detection System that converts routinely collected academic data into early, actionable insight. Using eight academic indicators — Attendance, Study Hours, Previous Score, Assignment Score, Internal Assessment, Practical Score, Participation and GPA — the system applies StandardScaler normalisation and a Multi-Layer Perceptron implemented in TensorFlow and Keras to classify students into four performance classes: At Risk, Average, Good and Excellent.

The use of ReLU activation in the hidden layers and a Softmax output layer, trained with Cross-Entropy Loss and the Adam optimizer, allows the model to learn non-linear relationships among academic indicators and to express its output as a probability distribution rather than a bare label. This makes it possible to report a confidence percentage and a corresponding risk level alongside every prediction, and to visualise how the probability is distributed across all four classes.

Delivered through an interactive Streamlit dashboard, the system requires no additional hardware and integrates with data that institutions already maintain. By shifting academic assessment from a retrospective to a predictive footing, it creates a practical window in which faculty can intervene while intervention still matters.

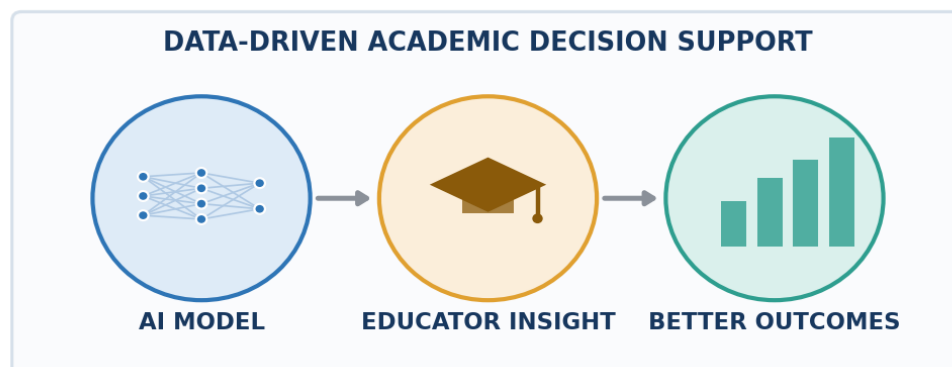


Figure 15: AI-Assisted Academic Decision Support

10.3 References

The following sources were used in preparing this report. They are the official documentation pages of the libraries and frameworks employed in the project.

- [1] TensorFlow Documentation. TensorFlow — An End-to-End Open Source Machine Learning Platform. Available at: <https://www.tensorflow.org/>
- [2] Keras Documentation. Keras — Deep Learning API for Python. Available at: <https://keras.io/>
- [3] Scikit-learn Documentation. Scikit-learn — Machine Learning in Python. Available at: <https://scikit-learn.org/stable/>
- [4] Scikit-learn Documentation. StandardScaler — Preprocessing and Feature Standardisation. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- [5] NumPy Documentation. NumPy — The Fundamental Package for Scientific Computing with Python. Available at: <https://numpy.org/doc/>
- [6] Pandas Documentation. Pandas — Python Data Analysis Library. Available at: <https://pandas.pydata.org/docs/>
- [7] Streamlit Documentation. Streamlit — A Faster Way to Build and Share Data Apps. Available at: <https://docs.streamlit.io/>
- [8] Python Software Foundation. The Python Language Reference. Available at: <https://docs.python.org/3/>